

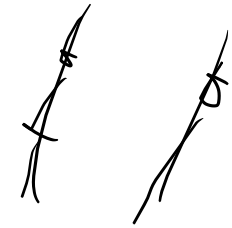
Ciklus levezetési szabálya

- [S] Atomi utasítás, megszakítás nélkül hajtjuk végre. S nem tartalmazhat ciklust vagy várakoztató utasítást.
- await β then S ta
Várakoztató utasítás. Amennyiben β igaz, az S program atomi utasításként a β kiértékelésével egyszerre hajtódik végre. S nem tartalmazhat ciklust vagy várakoztató utasítást. Ha β kiértékelhető de hamis, az utasítást tartalmazó folyamat blokkoltá válik; a végrehajtás nem tud továbblépni a várakoztató utasításon, amíg egy másik folyamat nem „segít” és változtatja meg az állapotot úgy hogy β teljesüljön.
- parbegin $S_1 \parallel \dots \parallel S_n$ parend vagy
cobegin $S_1 \parallel \dots \parallel S_n$ coend
Párhuzamos blokk. Befejeződik, ha minden komponensprogramja terminál. Végrehajtása valamely ágnek kiválasztását és az ott lévő komponens első utasításának, illetve a kapott maradék programnak egymás utáni végrehajtását jelenti.

Teljes helyesség leírása Hoare-hármassal

Amit eddig úgy jelöltünk hogy $Q \implies lf(S, R)$, jelölhetjük egy Hoare-hármassal is, a jelentésük ugyanaz: Q feltételnek eleget tevő állapotokból indulva garantált hogy az S program minden végrehajtása helyesen terminál, méghozzá olyan állapotban ahol R teljesül.

$$\{\{Q\}\}S\{\{R\}\}$$



- Atomi utasítás levezetési szabálya

$$\frac{\{\{Q\}\} S \{\{R\}\}}{\{\{Q\}\} [S] \{\{R\}\}}$$

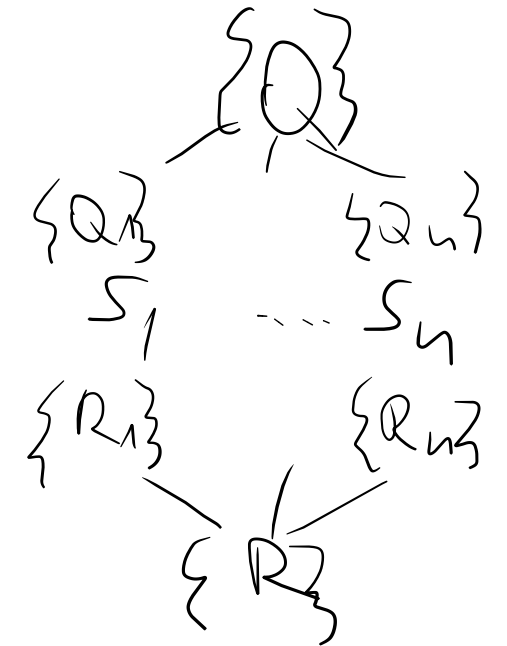
- Várakoztató utasítás levezetési szabálya

$$\begin{array}{l} 1) \quad Q \Rightarrow \beta \vee \neg \beta \\ 2) \quad \frac{\{\{Q \wedge \beta\}\} S \{\{R\}\}}{\{\{Q\}\} \text{await } \beta \text{ then } S \text{ ta } \{\{R\}\}} \end{array} \quad Q \wedge \beta \Rightarrow Q(S, R)$$

- Párhuzamos blokk levezetési szabálya

$$\begin{array}{l} 1) \quad Q \Rightarrow Q_1 \wedge \dots \wedge Q_n \\ 2) \quad \forall i \in \{1, \dots, n\} : \{\{Q_i\}\} S_i \{\{R_i\}\} \\ 3) \quad R_1 \wedge \dots \wedge R_n \Rightarrow R \\ 4) \quad \text{a párhuzamos blokk holtpontmentes} \end{array}$$

$$5 \quad \frac{\{\{Q_i\}\} S_i \{\{R_i\}\} \text{teljes helyességi tételek interferencia-mentesek}}{\{\{Q\}\} \text{parbegin } S_1 \parallel \dots \parallel S_n \text{parend } \{\{R\}\}}$$



Holtpont

```
S1{ a=0; b=0;  
  parbegin  
    S2{ while TRUE do  
        α0: ② a:=a+1;  
        α1: ③ await b ≠ 0 then  
        α2: a:=a+3  
        ta;  
      od  
      ||  
      Sn{ while TRUE do  
          β0: ① a:=2*a;  
          β1: ④ await a ≠ 1 then  
          β2: b:=b+1  
          ta;  
        od  
      }  
    parend  
  }
```

Holtpontban minden párhuzamos folyamat vagy befejeződött vagy blokkoltá vált.

és legalább egy blokkolt

(S):

⋮
A₁: await β₁ then S₁ ta;
⋮
A_i: await β_i then S_i ta;
⋮
T₁: parbegin S₁¹ || ... || S_{n₁}¹ parend
⋮
A_j: await β_j then S_j ta;
⋮
T_n: parbegin S₁ⁿ || ... || S_{n_n}ⁿ parend
⋮
A_m: await β_m then S_m ta;
⋮

pre(A_j)

post(S₁ⁿ)

•

$$D(S) = \left[\bigvee_{j=1}^m (pre(A_j) \wedge \neg B_j) \right] \vee \left[\bigvee_{k=1}^n \underline{D1(T_k)} \right]$$

Szélv fark

A formula azt fejezi ki hogy van benne (de nem párhuzamos blokkon belüli, hanem közvetlenül a „főprogramban”) blokkolt várakoztató utasítás, vagy valamely párhuzamos blokkja blokkolt.

•

$$\underline{D1(T_k)} = \left[\bigwedge_{j=1}^{n_k} (post(S_i^k) \vee D(S_i^k)) \right] \wedge \left[\bigvee_{i=1}^{n_k} \underline{D(S_i^k)} \right]$$

A T_k párhuzamos blokk blokkolt, ha minden S_i^k komponense blokkolt vagy véget ért, miközben van legalább egy komponense ami blokkolt.

Amennyiben D(S) azonosan hamis, nem lehet az S programnak olyan végrehajtása hogy holtpont előfordulhatna; S holtpontmentes.

2. $A = (x:\mathbb{Z})$

$$Q = (x=0)$$

$S_1:$

$\{x=0 \vee x=1\}$
 await $x=1$ then SKIP ta
 $\{x=1\}$

$S_2:$

$\{x=0\}$
 $x:=1$
 $\{x=1\}$

Bizonyítsuk be hogy $\{\{x=0\}\}$ parbegin $S_1 \parallel S_2$ parend $\{\{x=1\}\}$ teljesül. ✓

$$\checkmark 1) Q \Rightarrow Q_1 \wedge Q_2: x=0 \Rightarrow (x=0 \vee x=1) \wedge (x=0) \quad \checkmark$$

$$\checkmark 2) Q_2 \Rightarrow \ell_f(S_2, R_2): Q_2 \Rightarrow R_2 \quad (x=0) \Rightarrow (1=1) \quad \checkmark$$

$$Q_1 \Rightarrow \ell_f(S_1, R_1): \alpha) Q_1 \Rightarrow \beta \vee \neg \beta: (x=0 \vee x=1) \Rightarrow x=1 \vee x \neq 1 \text{ eldönthető} \quad \checkmark$$

$$\checkmark \beta) Q_1 \wedge \beta \Rightarrow \ell_f(S_1, R_1) \quad (x=0 \vee x=1) \wedge x=1 \Rightarrow (x=1) \quad \checkmark$$

$$\checkmark 3) R_1 \wedge R_2 \Rightarrow R: x=1 \wedge x=1 \Rightarrow x=1 \quad \checkmark$$

✓ 4) holapont mentesség

✓ 5) interferencia mentesség

2. $A = (x:\mathbb{Z})$

$$Q = (x=0)$$

Q_1

R_1

$S_1:$

$$\{x = 0 \vee x = 1\}$$

await $x = 1$ then SKIP ta

$$\{x = 1\} \quad \beta$$

Q_2

R_2

$S_2:$

$$\{x = 0\}$$

$x := 1$

$$\{x = 1\}$$

$$Q = (x=1)$$

Bizonyítsuk be hogy $\{\{x = 0\}\} \text{ parbegin } S_1 \parallel S_2 \text{ parend } \{\{x = 1\}\}$ teljesül.

$$\vee \vee \left(\left(x=1 \vee (x=0 \vee x=1) \wedge x \neq 1 \right) \wedge (x=1) \right) \wedge \left((x=0 \vee x=1) \wedge x \neq 1 \right)$$

*

*



erősebb HAMIS

2. $A = (x:\mathbb{Z})$

	$S_1:$
Q_1	$\{x = 0 \vee x = 1\}$
	await $x = 1$ then SKIP ta
R_1	$\{x = 1\}$ β

	$S_2:$
Q_2	$\{x = 0\}$
	$x := 1$
R_2	$\{x = 1\}$

Bizonyítsuk be hogy $\{\{x = 0\}\}$ parbegin $S_1 \parallel S_2$ parend $\{\{x = 1\}\}$ teljesül.

✓ 5) S_2 interferálhat S_1 -el?

✓ $Q_1 \wedge Q_2 \Rightarrow \neg \mathcal{I}(S_2, Q_1) : Q_1 \wedge Q_2 \Rightarrow Q_1^{x \leftarrow 1} : (x=0 \vee x=1) \wedge (x=0) \Rightarrow (1=0 \vee 1=1) \checkmark$ ← nem zavarja el

✓ $Q_1 \wedge Q_2 \Rightarrow \mathcal{I}(S_2, R_1) : R_1 \wedge Q_2 \Rightarrow R_1^{x \leftarrow 1} : (x=1) \wedge (x=0) \Rightarrow (1=1) \checkmark$

S_1 interferálhat S_2 -vel? nem ✓

$$j=1$$

A következő program biztosítja a kölcsönös kizárást: az S_1 és S_2 programok (folyamatok) nem tartózkodhatnak egyszerre a kritikus szakaszukban (mert nem használhatnak egyszerre egy közös erőforrást). Lássuk be hogy nem fordulhat elő holtponthelyzet, azaz hogy a folyamatok egymásra várnak.

```
{TRUE}
turn, flag1, flag2 := 1, false, false
{¬flag1 ∧ ¬flag2}
parbegin S1 || S2 parend
{FALSE}
```

$Inv = (turn = 1 \vee turn = 2)$
 $turn: \{1, 2\}$, míg $flag_1$ és $flag_2$ logikai változók.

```
{Inv ∧ ¬flag1}
S1:

{Inv ∧ ¬flag1}
while TRUE do
{Inv ∧ ¬flag1}
  non_critical_section_of_S1
  {Inv ∧ ¬flag1}
  flag1, turn := true, 1
  {Inv ∧ flag1}  $\beta_1$ 
  await (¬flag2 ∨ turn ≠ 1) then
  {Inv ∧ flag1 ∧ (¬flag2 ∨ turn = 2)}
    critical_section_of_S1
    {Inv ∧ flag1 ∧ (¬flag2 ∨ turn = 2)}
    flag1 := false
  {Inv ∧ ¬flag1}
  ta
{Inv ∧ ¬flag1}
od
{FALSE}
```

```
{Inv ∧ ¬flag2}
S2:

{Inv ∧ ¬flag2}
while TRUE do
{Inv ∧ ¬flag2}
  non_critical_section_of_S2
  {Inv ∧ ¬flag2}
  flag2, turn := true, 2
  {Inv ∧ flag2}  $\beta_2$ 
  await (¬flag1 ∨ turn ≠ 2) then
  {Inv ∧ flag2 ∧ (¬flag1 ∨ turn = 1)}
    critical_section_of_S2
    {Inv ∧ flag2 ∧ (¬flag1 ∨ turn = 1)}
    flag2 := false
  {Inv ∧ ¬flag2}
  ta
{Inv ∧ ¬flag2}
od
{FALSE}
```

$$\begin{aligned}
 & \left((turn = 1 \vee turn = 2) \wedge flag_1 \wedge \neg \beta_1 \right) \wedge \\
 & \left((turn = 1 \vee turn = 2) \wedge flag_2 \wedge \neg \beta_2 \right) = \\
 & \left((turn = 1 \vee turn = 2) \wedge flag_1 \wedge flag_2 \wedge \underbrace{turn = 1}_{*} \right) \wedge \\
 & \left((turn = 1 \vee turn = 2) \wedge flag_2 \wedge flag_1 \wedge \underbrace{turn = 2}_{*} \right) \\
 & \quad \quad \quad \downarrow
 \end{aligned}$$

hence